Highlights

A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing

Qi Yu, Wei Li

- Schedules are scored by their makespan after conflict-free AGV routing
- Dispatching trial-queries the reservation table for the earliest delivery
- Two speed-ups provably leave the chosen vehicle and the timetable unchanged
- Controlled instance families vary corridor contention intensity
- The closed loop is the main effect; dispatching adds a much smaller one

A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing

Qi Yu[a], Wei Li[b,*]

[a]*School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, China*
[b]*School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu, China*

Abstract

Flexible job shop scheduling with transport reads travel times from a constant matrix, assuming vehicles never interfere. When a finite AGV fleet shares a sparse corridor network, travel time becomes the outcome of conflict-free routing, and one job's corridor occupancy delays others. Where transport takes longer than processing, congestion concentrates on a few corridors and each operation may use arms of different efficiency, a slower but better-connected arm can beat a fast arm behind a congested corridor; prior work has not measured this trade-off inside the search objective. We propose a closed-loop bilevel framework: the upper level assigns machines and sequences operations, and the lower level plans conflict-free paths by time-window reservation. Routing-in-the-loop evaluation takes each candidate's fitness as its makespan after conflict-free routing, so yielding delay enters the search objective instead of a post-hoc check. Reservation-table-aware dis-

*Corresponding author.
Email addresses: 230240012@seu.edu.cn (Qi Yu), xchlw@seu.edu.cn (Wei Li)

patching trial-queries the transport legs on the current reservation table and selects the vehicle with the earliest delivery; two speed-ups provably change neither the selected vehicle nor the timetable. On 10 controlled instances (contention intensity 6.6% to 33.5%), with 10 seeds each, an equal wall-clock budget and corrected paired tests, the complete method reduces the realized makespan by 21.35% (94 wins, 5 losses) over the open-loop structure of the literature. Most comes from the closed loop, which alone cuts makespan by 18.86% and is significant on the vast majority of instances; dispatching is a much smaller effect, significant only when all instances are pooled, and the two do not amplify each other.

Keywords: Flexible job shop scheduling, Automated guided vehicles, Conflict-free routing, Bilevel optimization, Memetic algorithm, Vehicle dispatching

---

1. Introduction

1.1. Motivation and gap

Flexible job shop scheduling with transportation times conventionally reads travel times from a constant matrix indexed by location pairs (Xin et al., 2025), as recent models still do (Meng et al., 2025; Qin et al., 2026; Jafarian-Moghaddam et al., 2026). This is reasonable when vehicles rarely meet, but in real shops a few vehicles share sparse single-lane corridors, and at high fleet density their own traffic becomes the main cause of delay (Kim and Kim, 2026). Travel time then depends on each vehicle's path (conflict-free path planning, which we call routing) and on the order of reservations on contended segments, so waiting for occupied corridors lets one job's transport

delay the operations of others.

Machine flexibility leaves room to optimize this delay: when the processing time depends on the chosen arm, the assignment selects both the destination and the duration. On a constant matrix, choosing between a fast distant arm and a slower one near the load/unload point v_0 is arithmetic. Under contention, the waiting on the corridor to the fast arm is invisible to the matrix, so the slower, better-connected arm may be better, and the answer may change with the congestion level.

Figure 1 gives a recomputable instance with 4 jobs and 2 AGVs. The focal job J1 may use the fast arm M1 ($t^P = 3$), reached only through an exclusive trunk ($\tau = 6$) that three background jobs cross repeatedly, or the slow arm M2 ($t^P = 14$) on a separate branch. Alone and unimpeded, J1 needs 19 via M1 and 20 via M2, so the fast arm wins by the smallest possible margin, and the constant matrix gives instance makespans of 41 (M1) and 54 (M2). Under conflict-free routing, J1 on M1 occupies the trunk on both legs, J2 yields 17 at its entrance, and $C_{\max} = 95$; on M2, J2 yields only 1 and $C_{\max} = 60$. Only the evaluation has changed, yet the ranking reverses.

Most existing work covers only part of this setting: scheduling with transportation uses a constant matrix, and conflict-free path planning takes task origins, destinations and release times as given. Van Os and Basten (2026) formalize the combined problem and solve small benchmark instances to proven optimality. We focus on settings not previously taken as a dedicated object of study: transport longer than processing ($\bar{T}_t/\bar{T}_p \approx 4.0$, against at most 0.12 on common public instances), corridors exclusive over their whole length (van Os and Basten reserve nodes; Section 3.5), waiting only at berths,

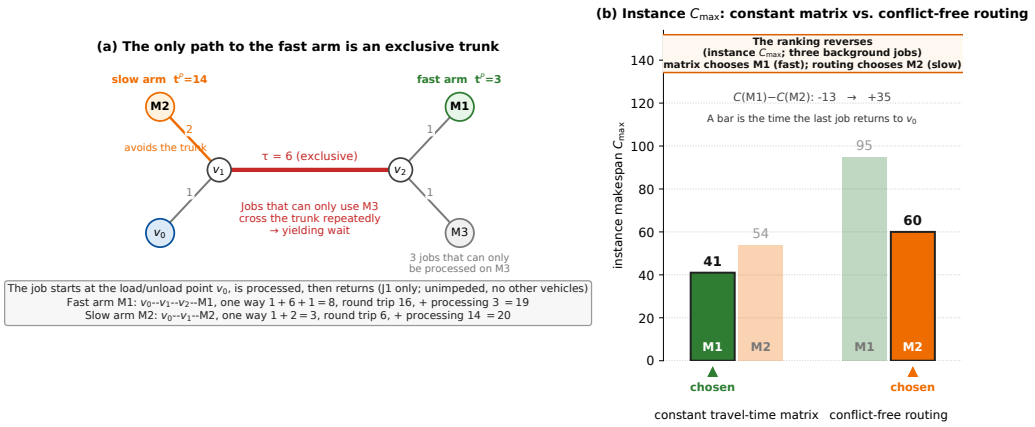


Figure 1: Motivating instance. (a) Network: exclusive trunk ($\tau = 6$) to the fast arm M1, short corridor to the slow arm M2, three M3-only jobs crossing the trunk. (b) Instance makespan $C_{\max}$ of the two assignments of J1 under the constant matrix and under conflict-free routing. Of 1200 nearby parameter combinations, 205 (17%) show the reversal.

and congestion on a few shared corridors, with contention intensity as a controlled variable and sizes that exact decomposition has difficulty solving. There, heuristics schedule first and resolve conflicts afterwards (Sun et al., 2023), feed back realized durations once without reassignment (Li et al., 2026), or embed routing in decoding without measuring its benefit and cost (Lyu et al., 2019). Under equal compute it is thus unmeasured how to trade off "fast but congested" against "slow but clear" arms, what counting yielding delay in the search gains, and at what contention this gain is worth its cost.

1.2. Approach and contributions

The upper level of our closed-loop bilevel framework decides machine assignment and operation sequencing; the lower level plans conflict-free paths by time-window reservation. On the evaluation path, routing-in-the-loop

evaluation ranks each candidate by its makespan after conflict-free routing, so yielding wait enters the search objective; such a decode costs over an order of magnitude more than one on unimpeded times, and neither Li et al. (2026), who list a fully iterative loop as future work, nor Lyu et al. report its cost or gain. On the decision path, reservation-table-aware dispatching trial-queries both legs for each candidate vehicle on the current reservation table, without writing it, and selects the earliest feasible delivery instead of idle-first dispatching (Lu et al., 2022); admissible lower-bound pruning and winner-path reuse skip only trial queries that cannot affect the result.

A four-level baseline ladder forms a 2×2 design over whether the search counts conflict-induced delay and whether vehicles are chosen by unimpeded time or by trial query: B0 is the open-loop structure of constant-matrix methods and Li et al. on a real network, B0+ dispatches by the reservation table on a frozen schedule as in Sun et al., B1 closes the loop with rule-based dispatching, and B2 is our method. All share the search, the router and the wall-clock budget, and are compared by realized makespan (Section 5.1.2). With dispatching fixed, the closed loop reduces makespan by 18.86% ($p < 10^{-4}$) and remains significant on 9 of the 10 instances after Holm correction; within it, reservation-table-aware dispatching adds 2.45% ($p = 0.0054$) but is significant on no single instance; end to end, the reduction is 21.35% ($p < 10^{-4}$), slightly below the 22.6% predicted by independent composition (Section 5.3). Our contributions are:

1. **Controlled instances.** Instance families with controllable contention intensity let us examine how high contention must be before counting yielding wait in the schedule brings a gain (Section 5.1).

2. **Routing-in-the-loop evaluation.** Its gain and cost over the open loop are measured at an equal wall-clock budget, and every chromosome decodes into a conflict-free, deadlock-free timetable without repair (Proposition 2).

3. **Low-overhead dispatching.** The speed-ups make each dispatching evaluation 2.33 times faster without changing the selected vehicle or the final timetable (Proposition 3), which makes a net gain possible at an equal budget.

4. **Decomposed evidence.** The closed loop is the larger effect and dispatching a much smaller one whose sign varies with the vehicle-to-arm ratio; the two do not amplify each other. An extra cost on occupied corridor–time slots (hereafter, surcharge) and yield-record-guided reassignment or staggering only add overhead; the failure of the surcharge suggests checking first whether the receiving level's objective already contains what an interface would pass.

Sections 2–5 cover related work, the problem (notation in Table 2), the framework and the experiments; the final section concludes.

2. Related work

Related work comprises scheduling with transportation, conflict-free routing with exogenous tasks, and their integration; Table 1 compares them with this paper.

2.1. Two strands

Flexible job shop scheduling with transportation (FJSP_PT) brings material movement into the schedule (FJSP review (Dauzère-Pérès et al., 2024); benchmarks (Homayouni and Fontes, 2021)), and its dominant convention is a constant travel-time matrix, a fixed $t(a, b)$ regardless of corridor occupancy (Xin et al., 2025). Recent work keeps the matrix and improves solution methods: Meng et al. (2025) close 29 open benchmark instances and improve 27 best-known solutions with constraint programming and a genetic algorithm; Qin et al. (2026) use Q-learning and a local search guided by AGVs and the critical path; Jafarian-Moghaddam et al. (2026) jointly optimize scheduling and internal logistics. Vehicles thus never delay one another, and although critical-chain local search is mature (Otala et al., 2021; Tamssaouet and Dauzère-Pérès, 2023), a critical path on the matrix leads back to an operation or a machine, not to a corridor; Xin et al. note that such assumptions "even change [the problem's] nature" and that $\bar{T}_t/\bar{T}_p$ is likely to affect algorithm performance.

Optimal multi-agent path finding (MAPF) is NP-hard (Gao et al., 2023), and Kasaura et al. (2022) extend Safe Interval Path Planning to continuous-time road networks. For continuously arriving tasks, Zhang et al. (2024b) optimize edge weights offline to steer agents away from bottlenecks, and Kim and Kim (2026) propose congestion-aware dispatching rules for large AGV fleets; surveys (Lu et al., 2022; Fragapane et al., 2021) treat dispatching and path planning as separate layers. Task release times and endpoints are inputs, so the router cannot revise the assignment, and congestion is measured indirectly (local density, offline edge weights), without identifying

which transport caused yielding, when, and where.

2.2. Integration and our position

Van Os and Basten (2026) use logic-based Benders decomposition, whose integer-programming routing subproblem returns conflicts to a constraint-programming master as cuts; they solve most benchmarks to proven optimality and improve about half by at least 10% over prior heuristics, but only at small scale, and list fuller use of "which conflicts caused the infeasibility" as future work. Sun et al. (2023) fix the schedule by a hybrid genetic algorithm and then resolve conflicts by time-window Dijkstra, so congestion found later cannot change the schedule. Li et al. (2026) feed realized durations back once, shifting start times of the current solution without reassignment, with one eligible machine per operation. Lyu et al. (2019) route inside genetic-algorithm decoding but study fleet sizing on small instances, with neither an open-loop comparison nor a cost report.

The gap is thus not whether conflicts are handled but how routing results enter the schedule and what that is worth. Beyond the reach of exact decomposition, and with corridors rather than nodes reserved, we route every candidate, dispatch by trial query, measure both contributions at an equal wall-clock budget, and vary contention intensity, since instances with the same $\bar{T}_t/\bar{T}_p$ may differ greatly in how much congestion reassignment can bypass.

Table 1: Related work versus this paper. Het.: machine-dependent processing times on optional machines; Excl.: one vehicle per corridor at a time; transport times are unimpeded (ignoring other vehicles) or routed (with yielding wait). Controls: Section 5.5.

Work class	Ref.	Het.	Excl.	Transport times used to evaluate schedules	Research focus
FJSP_PT metaheuristics	Meng et al., 2025; Qin et al., 2026; Jafarian-Moghaddam et al., 2026	yes	no	unimpeded (matrix)	quality under a matrix
Lifelong MAPF	Gao et al., 2023; Zhang et al., 2024b	—	yes	none (tasks exogenous)	throughput
Congestion-aware dispatching	Kim and Kim, 2026	—	no	none (tasks exogenous)	large-fleet congestion
Two-stage FJSP + routing	Sun et al., 2023	yes	yes	unimpeded; routed after freezing	schedule, then resolve
RL scheduling + PBS	Li et al., 2026	no	yes	unimpeded; one correction	RL with multi-vehicle paths
GA + time-window routing	Lyu et al., 2019	yes	yes	routed, every candidate	fleet size
Exact LBBD	van Os and Basten, 2026	yes	no	bound + conflict cuts	small-scale optimality
This paper		yes	yes	routed, every candidate; trial-query dispatching	fast/slow-arm trade-off; closed-loop gain and cost

3. Problem statement and model

3.1. Problem statement

An instance consists of n jobs, N_M robotic arms, and N_A vehicles (AGVs) on a shop-floor graph (Table 2). Job j is a fixed chain of n_j operations $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} may be processed on any arm in its option set $\Omega_{ji} \subseteq M$, taking an arm-dependent time t^P_{jim}. Arms occupy fixed nodes, and the fleet moves jobs between them. Four decision groups are solved at once: (i) assign each operation to an arm in Ω_{ji}; (ii) sequence the operations on each arm; (iii) assign the induced transport tasks to vehicles and sequence each vehicle's tasks; (iv) plan for every move a path that is conflict-free in time and space, waiting allowed. The objective is to minimize the makespan $C_{\max}$.

Table 2: Notation.

Symbol	Meaning
$J = \{1, \ldots, n\}$, O_{ji}	jobs; i-th operation of job j, $i \le n_j$
$M = \{1, \ldots, N_M\}$, $\Omega_{ji} \subseteq M$	robotic arms; option set of O_{ji}
t^P_{jim}	processing time of O_{ji} on arm $m \in \Omega_{ji}$
$K = \{1, \ldots, N_A\}$	homogeneous unit-load vehicles
$G = (V, E)$, τ_e	shop-floor graph; traversal time of corridor e
v_0, $\mathrm{loc}(m)$	load/unload point; node of arm m
$t^*(a, b)$	shortest-path time $a \to b$ in G, ignoring contention
$\delta_{\mathrm{ret}} \in \{0, 1\}$	finished job returns to v_0 (baseline 1)
S_{ji}, C_{ji}, A_{ji}, $C_{\max}$	start, completion, material arrival; makespan

3.2. Modelling assumptions

A1. Static instance and complete information. All n jobs are at v_0 at $t = 0$; no release or due dates. Processing data, G, and vehicle states are known at planning time.

A2. Jobs. Fixed precedence chain per job. Processing and transport are non-preemptive; jobs cannot be split; no exogenous precedence among jobs.

A3. Robotic arms: partial flexibility and heterogeneous times. O_{ji} may run on any $m \in \Omega_{ji}$ (most operations have $|\Omega_{ji}| \geq 2$; $\Omega_{ji} = M$ is a special case). Times t^P_{jim} vary with the arm and need not be proportional across arms. An arm processes one operation at a time and does not fail; sequence-dependent setup is folded into t^P_{jim}.

A4. Buffers and loading. Ample input/output buffers and storage at v_0, so no blocking. Loading and unloading are negligible and do not occupy the arm.

A5. Vehicles. N_A homogeneous unit-load vehicles at constant speed, empty or loaded; no acceleration or battery. All stand empty and available at v_0 at $t = 0$. A task is a one-way move—v_0 to the first arm, between successive distinct arms, or, when $\delta_{\text{ret}} = 1$, last arm back to v_0; any vehicle may take any task, so a job's moves may be relayed across vehicles. After unloading, a vehicle is not forced back to v_0; it follows its own task chain, and routing may postpone its departure.

A6. Road network and conflict-freedom. G is strongly connected; each τ_e is a known constant. Corridors are *exclusive*: both directions of e share one reservation, so at most one vehicle occupies e at a time (no head-on conflict, no overtaking). Occupancy is half-open, $[t,\, t + \tau_e)$, so a second vehicle may enter at $t + \tau_e$. Nodes have ample berths, separate from the traversal resource; waits occur at berths and are always

permitted.

A7. Out of scope. Machine and vehicle failures, finite buffers and blocking, battery charging, and dynamic job arrivals (Gui et al., 2023; Zhao and Fu, 2026) are left to future work (item (5) of Section 6.2).

A1, A2, A4, and A5 are standard in FJSP_PT (Xin et al., 2025); our problem is defined by A3 and A6, which make transport duration an output of routing rather than a constant matrix entry. Deadlock-freedom is not assumed: Proposition 2 uses A4 and A6 to prove that any assignment and sequence decodes into a conflict-free, deadlock-free timetable, so the search need not repair infeasible solutions.

3.3. Decision variables and induced transport tasks

Let $x_{jim} \in \{0,1\}$ indicate that O_{ji} is assigned to arm m; write $\mathrm{m}(j,i)$ for the chosen arm and $d_{ji} = \mathrm{loc}(\mathrm{m}(j,i))$ for its node. When $\delta_{\mathrm{ret}} = 1$, a zero-duration dummy operation O_{j,n_j+1} at v_0 represents the return, so return moves share the task notation and the makespan is always the completion of each job's last operation. Unlike in vehicle routing, transport tasks are not an input but are *induced* by the assignment:

$$\mathcal{T}(x) = \Big\{ (j,i) \ : \ d_{j,i-1} \neq d_{ji} \Big\}, \qquad d_{j0} \equiv v_0, \tag{1}$$

where task (j,i) moves job j from $d_{j,i-1}$ to d_{ji} and is ready at $r_{ji} = C_{j,i-1}$ ($C_{j0} = 0$). Decision (i) thus fixes the number and endpoints of tasks: a job kept on one arm needs two moves, one that changes arm at every operation $n_j + 1$. Tasks are assigned to vehicles by $y_{tk} \in \{0,1\}$ with $\sum_{k \in K} y_{tk} = 1$ for each $t \in \mathcal{T}(x)$, and each vehicle's tasks are sequenced. A constant travel-time

matrix model shares this dependence but treats move durations as constants; here the assignment also determines on which corridors the moves contend.

3.4. Timing relations and feasibility

With $p_{ji} = \sum_{m \in \Omega_{ji}} x_{jim} t^P_{jim}$, assignment, precedence, and unit arm capacity give

$$\sum_{m \in \Omega_{ji}} x_{jim} = 1, \qquad \forall j, i, \quad (2)$$

$$C_{ji} = S_{ji} + p_{ji}, \qquad \forall j, i, \quad (3)$$

$$S_{ji} \;\geq\; A_{ji}, \qquad \forall j, i, \quad (4)$$

$$S_{j'i'} \;\geq\; C_{ji} \quad \vee \quad S_{ji} \;\geq\; C_{j'i'} \qquad \forall (j,i) \neq (j',i') \text{ on the same arm}, \quad (5)$$

where A_{ji} is the time at which the job actually appears at d_{ji}: $A_{ji} = C_{j,i-1}$ if $(j,i) \notin \mathcal{T}(x)$; otherwise, with a^{E}_{ji} the time at which the carrying vehicle reaches the pickup node after its empty leg,

$$\ell_{ji} = \max\big(a^{\mathrm{E}}_{ji},\; r_{ji}\big), \quad A_{ji} = \text{arrival time of the loaded leg departing at } \ell_{ji}, \quad (6)$$

so either may wait for the other. Each leg, empty or loaded, is a walk $(e_1, \ldots, e_q)$ in G whose corridor entry times satisfy

$$\eta_1 \;\geq\; \sigma, \qquad \eta_{l+1} \;\geq\; \eta_l + \tau_{e_l}, \quad l = 1, \ldots, q-1, \quad (7)$$

where σ is the earliest start of the leg; a strict inequality is a wait at a berth, and the leg ends at $\eta_q + \tau_{e_q}$. Any two distinct occupancies of the same corridor e, regardless of direction and vehicle, satisfy

$$\big[\eta,\; \eta + \tau_e\big) \;\cap\; \big[\eta',\; \eta' + \tau_e\big) \;=\; \emptyset, \quad (8)$$

the only constraint that couples jobs through space rather than through a shared arm. The objective is

$$\min\ C_{\max}, \qquad C_{\max} = \max_{j \in J}\ C_{j,\, n_j + \delta_{\mathrm{ret}}}. \tag{9}$$

In a semi-active schedule, Eqs. (4) and (5) tighten to

$$S_{ji} = \max\bigl(A_{ji},\ F_{\mathrm{m}(j,i)}\bigr), \tag{10}$$

where F_m is the completion time of the operation immediately before O_{ji} on arm m; Section 4 uses this to attribute a postponed start to waiting for the workpiece or for the arm.

3.5. Relation to FJSP_PT

Without Eq. (8), every leg takes an unimpeded shortest path, durations collapse to the constant matrix $t^*(a, b)$ (later a lower bound and the open-loop arrival estimate), and the model becomes flexible job shop scheduling with transportation (FJSP_PT); corridor exclusivity is the only difference. van Os and Basten (2026) reserve nodes, whereas we reserve edges in both directions; Lyu et al. (2019) also reserve edges but allow one vehicle per node, whereas our nodes have ample berths (like the standby areas beside real workstations), so congestion occurs only on corridors. Why the instances cannot be compared directly is limitation (1) in Section 6.2.

3.6. Computational complexity

Proposition 1. *The problem defined by Eqs.* (2)–(9) *is strongly NP-hard, and it remains so when every operation satisfies* $\Omega_{ji} = M$.

Proof. Set $\tau_e = 0$ for all e, give each arm a dedicated corridor from v_0, and let $N_A = n$: Eq. (8) then holds trivially and $A_{ji} = C_{j,i-1}$, leaving Eqs. (2)–(5), the flexible job shop with unrelated machines (Dauzère-Pérès et al., 2024). Its special cases $|\Omega_{ji}| = 1$ (classical job shop, three machines) and $\Omega_{ji} = M$, $n_j = 1$ (unrelated parallel machines, machine count in the input) are strongly NP-hard, and each restriction is a polynomial reduction. □

Hardness thus holds even without contention; fixing decisions (i)–(iii) still leaves conflict-free multi-agent routing (Gao et al., 2023), so routing adds to the scheduling difficulty rather than replacing it. Logic-based Benders decomposition solves most small benchmark instances of this setting to optimality (van Os and Basten, 2026); we target larger sizes.

3.7. Instance features

Size is fixed and only structure varies (size: limitation (1), Section 6.2). Four standard features are recorded: $\bar{T}_t/\bar{T}_p$, mean shortest-path travel between arm nodes over mean processing time, likely to affect performance (Xin et al., 2025); heterogeneity H, the coefficient of variation of each operation's processing times over its optional arms, averaged (singletons count 0), where larger H leaves less room to reassign around congestion, so we do not expect the gain to rise with H (Section 5.4); flexibility $F = \text{mean}_{ji} |\Omega_{ji}|/N_M$, the scope of reassignment; and N_A/N_M, small values favouring a transport bottleneck. Two are specific to road networks: contention intensity, the share of travel time that is yielding wait on the initial random population, which orders the instances (Table 4); and the funnel width $|\text{cut}(v_0)|$, the size of a smallest corridor set separating v_0 from every arm, which bounds how many

vehicles can leave v_0 at once (Family A varies it; `funnel` halves the value of `high`).

The lower bound $\mathrm{LB} = \max(\mathrm{LB}_{\mathrm{chain}}, \mathrm{LB}_{\mathrm{load}}, \mathrm{LB}_{\mathrm{cut}})$ combines a per-job chain bound (shortest inbound and return trips plus $\sum_i \min_m t^P_{jim}$), a load bound $\sum_{j,i} \min_m t^P_{jim}/N_M$, and a cut bound spreading the $X = 2n$ crossings of $\mathrm{cut}(v_0)$ (when $\delta_{\mathrm{ret}} = 1$) over its corridors:

$$C_{\max} \;\geq\; X \Big/ \sum_{e \in \mathrm{cut}(v_0)} \tau_e^{-1}. \tag{11}$$

$\mathrm{LB}_{\mathrm{cut}}$ is largest on eight of our ten instances and $\mathrm{LB}_{\mathrm{chain}}$ on the other two; the bound is loose (gap 27% to 58% to the best known solution) and checks only the order of magnitude, not near-optimality.

4. Closed-loop bilevel framework

4.1. Overview: routing-in-the-loop evaluation and the decision path

The framework assigns the four decision groups to two levels (Figure 2). A memetic algorithm (Zhang et al., 2024a) searches over assignment and sequencing; an event-driven decoder turns each candidate into transport tasks; and the lower level routes them conflict-free by time-window reservation, returning an executable timetable and its makespan. Unlike two-stage or single-correction schemes, the framework differs in how routing enters the schedule: the content and direction of the feedback, and whether every candidate receives it. On the *evaluation path*, i.e. routing-in-the-loop evaluation, every candidate's fitness is its makespan after conflict-free routing, yielding wait included. On the *decision path*, reservation-table-aware dispatching picks each vehicle by a trial query on the reservation table rather than by

unimpeded travel time. Section 5 measures the two paths separately and reports control results for two further inter-layer interfaces (Section 4.6): the surcharge on occupied corridor–time slots and the yield record.

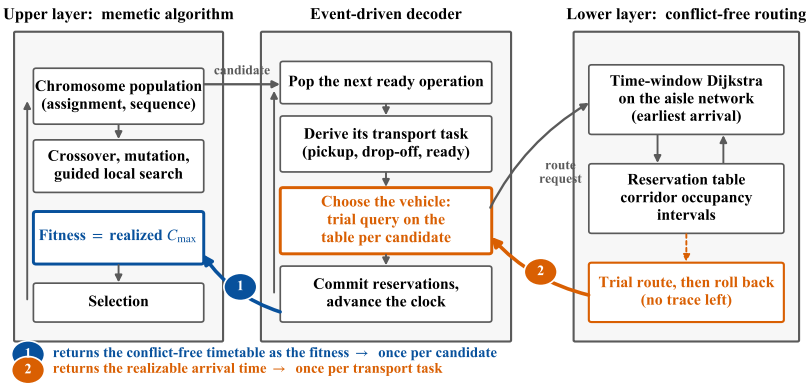


Figure 2: The closed-loop bilevel framework: memetic search (upper), event-driven decoder (middle), and time-window reservation routing (lower). Path 1 (blue) returns the conflict-free timetable as fitness, once per candidate; path 2 (orange) runs a two-leg trial query before each vehicle choice, once per transport task, without writing the final timetable. The ladder of Section 5 is arranged by which paths are open.

4.2. Lower level: time-window reservation and conflict-free routing

The reservation table maps each corridor, the only contended resource (A6), to a time-ordered list of half-open occupancy intervals $[t,\ t+\tau_e)$ labelled by vehicle and task. It supports four operations: earliest feasible entry no earlier than a given instant; write; revoke a task; and checkpoint and rollback around a trial query. Nodes take zero time and waits occur at berths, so nodes are not reserved, and both the conflict-free path planner (hereafter, the router) and an independent checker inspect only corridor occupancy.

The router is a time-window Dijkstra search (Algorithm 1) whose label (u, t), with cost t, means standing at the berth of u at time t. For each

corridor leaving u the table returns the earliest feasible entry $t' \geq t$, so the berth wait $t' - t$ of Eq. (7) is produced by the search itself. Written paths are not revoked within a decode, so vehicles are planned in task-generation order and a later-planned vehicle yields to an earlier one.

Lemma 1 (One earliest-arrival label per node suffices)**.** *Keeping only the earliest-arrival label at each node is sufficient to obtain a path that reaches the destination earliest under the current reservation table.*

Proof. Ample berths occupy no corridor (A6), so a vehicle at u since t can wait until any $t' > t$ and take every continuation open to one arriving at t'; an earlier label is never worse. Later queries see no more free slots, so $t \mapsto R.\textsc{EarliestEntry}(e, t, \tau_e) + \tau_e$ is non-decreasing; labels leave the queue in increasing time, so the first extraction of the destination is the earliest arrival. □

Ordinary Dijkstra therefore suffices, without the per-interval labelling of Safe Interval Path Planning (SIPP) (Kasaura et al., 2022); a second criterion such as a surcharge would require a multi-label search (Section 4.6).

Algorithm 1 ROUTE: conflict-free planning of a single leg

Require: s, g, departure t_0, table R, flag *commit*
push (s, t_0) onto a queue keyed by (time, node id)
while the queue is nonempty **do**
    pop the least label (u, t)
    if $u = g$ **then**
        backtrack occupancies; write them if *commit*; **return** t
    end if
    for all corridors $e = (u, v)$ **do**
        $t_{\text{in}} \leftarrow R.\textsc{EarliestEntry}(e, t, \tau_e)$; update $(v,\ t_{\text{in}} + \tau_e)$ if strictly earlier
    end for
end while
return failure ▷ unreachable; see text

A path always exists because reservations are finite and G is strongly connected: at worst the vehicle waits until every occupancy has ended, then takes an unimpeded shortest path. With W the mean number of intervals per corridor, a call costs $O\big(|E|\,(W^2 + \log|V|)\big)$ ($O(W)$ candidate entries per corridor, each tested in $O(W)$); a logarithmic earliest-entry query would give $O\big(|E|(\log W + \log|V|)\big)$, but we use the linear scan. On an empty table the search yields t^*, used for open-loop estimates, the reassignment proximity term, the two-stage baseline, and the pruning bound (Section 4.5).

4.3. Upper level: encoding and event-driven decoding

An individual has two segments: a machine-assignment vector whose k-th entry is the arm chosen from Ω_{ji} for the k-th of the N operations, and an operation sequence in which job j appears n_j times (plus once for its dummy when $\delta_{\text{ret}} = 1$), its x-th appearance standing for O_{jx}; operations sharing an arm are ordered by the sequence. Vehicles are chosen at decode time (Section 4.4), since a third segment would enlarge the search space and lose guaranteed decodability.

The decoder (Algorithm 2) implements the evaluation path. Sweeping the sequence, it generates a task of Eq. (1) whenever an operation's arm differs from the job's location, chooses a vehicle, and plans the empty and loaded legs against the live table, joined by Eq. (6). The operation starts by Eq. (10), and the recorded `bind` field (waiting for the workpiece or for the arm) is authoritative for later tracing of postponed starts.

Decoding is deterministic: reservations are written in task-generation order, the routing queue breaks ties by node id and updates a label only on a strictly earlier arrival, and dispatching scans vehicles by increasing id and

Algorithm 2 DECODE: fitness from a complete conflict-free simulation

Require: assignment x, operation sequence

$F_m, \text{ready}_j, \text{avail}_k \leftarrow 0$; $\text{pos}_j, \text{loc}_k \leftarrow v_0$; clear R
for all O_{ji} in sequence order (dummies included) **do**
  $d \leftarrow \text{loc}(m)$ for the arm m of O_{ji} under x
  if $\text{pos}_j = d$ **then**
    $A_{ji} \leftarrow \text{ready}_j$
  else
    choose vehicle k (Section 4.4); $a^{\text{E}} \leftarrow \textsc{Route}(\text{loc}_k, \text{pos}_j, \text{avail}_k, R, \text{true})$
    $A_{ji} \leftarrow \textsc{Route}(\text{pos}_j, d, \max(a^{\text{E}}, \text{ready}_j), R, \text{true})$; $\text{loc}_k, \text{avail}_k \leftarrow d, A_{ji}$
    accumulate yielding time by corridor
  end if
  `bind` $\leftarrow$ `machine` if $F_m > A_{ji}$, else `arrive`
  $S_{ji} \leftarrow \max(A_{ji}, F_m)$; $C_{ji} \leftarrow S_{ji} + p_{ji}$; $F_m, \text{ready}_j \leftarrow C_{ji}$; $\text{pos}_j \leftarrow d$
end for
return $\max_{j,i} C_{ji}$, timetable, yield statistics

keeps the first strictly better one. A chromosome thus maps to a unique timetable, on which the seed-paired, equal-budget comparisons of Section 5 depend.

Proposition 2 (Every chromosome is decodable)**.** *For any machine assignment and any operation sequence, Algorithm 2 terminates and returns a conflict-free, deadlock-free timetable of finite makespan. The search therefore need not invoke a repair operator.*

Proof. The sequence is a topological order of each job's chain, so ready_j is known when read, and ample buffers (A4) let Eq. (10) give a finite start once the job has arrived. Each ROUTE call terminates by the existence argument after Algorithm 1 and occupies only free slots, so Eq. (8) holds by construction. Deadlock cannot occur: the corridor is the only contended resource (A6), waits occupy only ample berths, and every written occupancy ends at a finite time, so no circular wait arises. The sweep runs $N + n\delta_{\text{ret}}$ times, each scheduling finitely many events at finite times. □

4.4. Reservation-table-aware dispatching

The open-loop rule scores each vehicle by its unimpeded-shortest-path delivery estimate,

$$\text{score}(k) = \max\big(\text{avail}_k + t^*(\text{loc}_k, \text{pickup}),\, \text{ready}_j\big) + t^*(\text{pickup}, \text{destination}), \tag{12}$$

the idle-first dispatching of Li et al. (2026) and a typical single-attribute rule in multi-AGV dispatching (Lu et al., 2022); ignoring the reservation table, it may pick a nearby vehicle whose corridors are already occupied. The closed-loop default, reservation-table-aware dispatching, runs a trial query per candidate vehicle: the empty leg is planned and temporarily written (otherwise both legs could share a corridor–time slot and the result would be too early), the loaded leg is planned against that state, the earliest feasible delivery is recorded, and the table is rolled back. The vehicle with the earliest feasible delivery is selected; since vehicles are identical (A5) and differ only in location and corridor occupancy, contention on the table can directly determine the choice. The cost is up to $2N_A$ instead of 2 routing calls per task, about 12.2 times the evaluation cost of rule-based dispatching; unless reduced (Section 4.5), this overhead offsets the makespan gain under an equal wall-clock budget, so Section 5 compares the two rules only under the same wall-clock limit.

4.5. Low overhead: admissible lower-bound pruning and winner-path reuse

At equal generations reservation-table-aware dispatching beats rule-based dispatching by 2.90%, but at equal wall-clock time its 12.2-fold evaluation cost leaves a net effect close to zero; the two speed-ups below reduce this multiple without changing any selection.

Admissible lower-bound pruning.. Yielding can only delay routing, never beat the shortest path that ignores contention, so the open-loop estimate of Eq. (12) is an admissible lower bound on the realized delivery time:

$$\begin{aligned} \mathrm{lb}(k) = \; & \max\big(\mathrm{avail}_k + t^*(\mathrm{loc}_k, \mathrm{pickup}),\, \mathrm{ready}_j\big) \\ & + t^*(\mathrm{pickup}, \mathrm{destination}) \;\le\; \mathrm{est}(k), \end{aligned} \tag{13}$$

where $\mathrm{est}(k)$ is the earliest feasible delivery from a complete two-leg trial query for vehicle k. A vehicle whose bound is no earlier than the best trial-query delivery so far needs no routing call.

Winner-path reuse.. Rollback discards the winner's two legs, which the decoder would replan on the same restored table; as routing is deterministic, the cached trial-query paths are written directly, saving two calls.

Proposition 3 (The two speed-ups do not change the selected vehicle or the timetable)**.** *In the implementation, vehicles are queried in increasing id order, and the selected vehicle is replaced only when some vehicle's feasible delivery is strictly earlier than the current best. Under this rule, after the pruning of Eq.* (13) *and winner-path reuse are added, the selected vehicle, the occupancies written to the reservation table, and the final timetable are the same as when a complete two-leg trial query is run for every vehicle.*

Proof. If $\mathrm{lb}(k) \ge \mathrm{est}^*$, the current best, then $\mathrm{est}(k) \ge \mathrm{lb}(k) \ge \mathrm{est}^*$ by Eq. (13), so k is not strictly earlier and would not be selected anyway; pruned vehicles write nothing to the table, which is restored after each trial query, so later queries are unaffected. The winner's legs were computed against the very table state the decoder writes into after restoration, and routing is deterministic, so cached and replanned paths give identical occupancies. □

Jointly, measured as one on/off switch (Table 9) and not attributed to pruning or reuse separately, the speed-ups give a 2.33-fold speed-up, cut routing calls by 47.4%, and reduce the cost multiple from 12.2 to 5.23. Trial-query cost grows roughly linearly with N_A, and pruning weakens as the fleet grows yet still cuts 42.2% of the calls on the largest fleet tested; the gain may thus vanish beyond some fleet size, which the present number of seeds is not sufficient to determine (Section 5.4).

4.6. Two further inter-layer interfaces: surcharges and yield records

Both paths read realized durations, i.e. traversal times after conflict-free routing, for every candidate, whereas schedule-then-route methods obtain them once, after the assignment is fixed. Realized durations do not reveal which decision caused a wait, so we implement two richer interfaces and compare them under an equal wall-clock budget (Section 5.5).

A *surcharge* $\pi(e, b)$, by analogy with shadow prices in Lagrangian relaxation and column generation, is added to arrival time by a vehicle occupying corridor e in slot b: routing minimizes arrival time $+\,\theta\cdot$ the surcharges traversed (the surcharge coefficient $\theta \geq 0$ is the only tunable parameter), and reassignment likewise pays for its traversals. π is estimated by perturbing one slot's capacity and re-solving, or by a low-cost approximation refreshed each generation. While fitness comes from a complete conflict-free decode, such a cost adds no information: contention wait already enters arrival time, makespan (Eq. (10)) and fitness, so the surcharge counts it twice, and blocking of later tasks already shows in the decoded makespan; we verify this inference on this one interface only. By Lemma 1, the second criterion also forces a multi-label Pareto search, raising evaluation cost whether or not the

surcharge is non-zero.

A *yield record* instead returns structure: the triple (corridor, time slot, preceding occupying transport) that routing knows at each yield. It enters no objective, so no delay is double-counted; it only shapes the neighbourhood of yield-record-guided local search, applied with the same setting to the top-ranked individuals of each generation in all four main-batch configurations. The search backtracks from the makespan-determining event along a critical chain that includes transport legs, since a wait for the workpiece may stem from an upstream operation, a missing vehicle, or contention; links are labelled as in Table 3, a yield on a loaded leg counting as `corridor` and an empty leg classified by whether yielding exceeds half its duration. For each of at most $L = 5$ real operations on the chain, reassignment yields one neighbour by moving it from arm m to the arm $m' \in \Omega_{ji} \setminus \{m\}$ minimizing

$$\text{score}(m') = t^*\big(d_{j,i-1}, \text{loc}(m')\big) + t^P_{jim'}, \tag{14}$$

trading a longer processing time for a shorter path with better traffic (cumulative yield, a fleet- and horizon-wide total, is not commensurable and is omitted). Staggering advances the blocked operation or postpones the preceding occupier on the longest `corridor` link by swapping adjacent appearances of different jobs, preserving decodability (Proposition 2). The first improvement among these at most $L + 2 = 7$ neighbours is accepted, for at most three rounds with the chain recomputed each round; the same tracing reads the case study (Section 5.7).

Table 3: Critical-chain link labels and remedies. Only `corridor` names a corridor, time slot, and preceding transport.

Label	Criterion	Remedy
`operation`	a real operation occupies its arm	—
`machine`	waiting for the arm	resequence
`upstream`	vehicle first, material not ready	recurse to $O_{j,i-1}$
`vehicle`	material ready, vehicle late, little yielding	change vehicle
`corridor`	yield on a given corridor and slot	reassignment or staggering

4.7. Outer genetic algorithm, complexity and cost

The outer layer is a conventional genetic algorithm: a random initial population plus a fastest-arm and a greedy load-balancing individual; tournament selection with a small elite; precedence-preserving order crossover on the sequence, uniform crossover on the assignment; and swap or reassignment mutation (hyperparameters in Section 5.1.3).

One evaluation processes $N + n\delta_{\text{ret}}$ events with at most $2N_A$ routing calls per transport under closed-loop dispatching, so an individual costs $O\big((N + n)\, N_A\, |E|\, (W^2 + \log |V|)\big)$; open-loop selection drops the factor N_A. The speed-ups leave this bound and every selection unchanged (Proposition 3) but cut routing calls by 47.4% (42.2% to 61.2% across instances), in effect shrinking N_A; local search adds at most $3(L + 2) = 21$ decodes per selected individual. On the ladder batch one evaluation takes 0.38 ms open-loop, 2.70 ms with rule-based dispatching, and 14.2 ms full closed-loop (about 37 times open-loop); in an earlier scan the surcharge raised it from 15.05 ms to 72.9 ms (about 4.8 times), so costs span about two orders of magnitude, although the two batches' timings are not directly comparable. A costlier evaluation buys fewer evaluations, so whether an interface pays off must be

measured, and Section 5 aligns wall-clock time rather than generations.

These results guarantee correctness, not solution quality: Proposition 2, Lemma 1, and Proposition 3 ensure that every candidate is feasible, that its evaluation matches the conflict model of Section 3, and that the speed-ups change no selection, and deterministic decoding makes the procedure reproducible. As with most metaheuristics, solution quality has no theoretical bound and is evaluated experimentally in the next section.

5. Experiments

This section measures the gains of the closed loop and of reservation-table-aware dispatching and whether they amplify each other, asks on which instances they hold and whether the present seeds can resolve this, and explains why the other interfaces yield no net gain.

5.1. Experimental setup

5.1.1. Controlled instance families

FJSP_PT benchmarks write travel times as a matrix and supply no corridor network. Of the 7 layouts of Homayouni and Fontes (2021), 7 matrices are entirely asymmetric, ruling out undirected corridors, and 2 also violate the triangle inequality, which no graph's shortest paths do; a network reverse-engineered from them would be a different instance. We therefore generate controlled instances on an explicit road network, so that each feature of Section 3.7 can be varied on its own; public instances, with contention removed, calibrate only the upper-level search (Section 5.8).

All instances share the core of Table 4, with $\bar{T}_t/\bar{T}_p$ scaled so that vehicle

contention is the main constraint and H standardized per operation so that F does not change it.

Family A: layout. In a spine layout a main corridor (the spine) runs through the arm area and each arm hangs off it on its own branch corridor, as in high-volume plants where congestion on the central corridor is a known problem (Suh and Choi, 2022). Placing v_0 at one end of the spine, rather than where trips are balanced as real shops do, sends every transport through the same exit and corridor stretch: a deliberately high-contention construction, and exactly the case the closed loop targets. Parallel lanes of equal duration change capacity, not distance; a tandem single-vehicle loop (Rahimikelarijani et al., 2020) has almost no contention. `funnel`, `high`, and `mid` differ only in their lanes; `funnel` and `high` differ by one exit at v_0, halving the minimum cut, a single-factor contrast. The grids `low` and `scatter` (arms at the farthest points) cover the low-contention range, but their farthest unimpeded trip is 18.0 against 10.0, so only differences among the spine instances are attributable to capacity. Contention intensity (31.9% to 6.6%) is not monotone in this order; the paper-wide maximum 33.5% is the $F = 1.0$ instance of Family C.

Family B varies the vehicle-to-arm ratio on `funnel` to test whether dispatching has an upper limit in fleet size; *Family C* varies flexibility, the fraction of arms each operation may use. Their shared base point `A funnel` is run once, so $5 + 3 + 2 = 10$, and must be counted when reading Families B and C. Contention intensity is measured on the initial random population because optimized solutions route around congestion and erase the differences among instances. Relative to the 20 public instances, our base F is

about one and a half times the median 0.38, H exceeds the maximum 0.258 (median 0.135), and $\bar{T}_t/\bar{T}_p$ deliberately exceeds the maximum 0.12; whether the high H inflates the closed-loop gain is tested in Section 5.4.

Table 4: Controlled instances. Shared core: 12 jobs, 36 real operations, 8 arms, $H = 0.3$, $\bar{T}_t/\bar{T}_p \approx 4.0$. Contention intensity: share of yielding wait in travel time on the initial random population.

Instance	Varied factor	Nodes	Corridors	N_A	F	Contention	Composite LB (time units)
A funnel	layout	13	12	12	0.61	31.9%	24.0
A high	layout	14	14	12	0.61	30.9%	17.0
A low	layout	16	24	12	0.61	11.8%	36.0
A mid	layout	15	16	12	0.61	10.8%	17.0
A scatter	layout	16	24	12	0.61	6.6%	36.0
B NA/NM 0.5	N_A/N_M	13	12	4	0.61	15.8%	24.0
B NA/NM 1.0	N_A/N_M	13	12	8	0.61	30.2%	24.0
B NA/NM 2.0	N_A/N_M	13	12	16	0.61	32.4%	24.0
C F=0.3	flexibility	13	12	12	0.30	27.7%	24.0
C F=1.0	flexibility	13	12	12	1.00	33.5%	24.0

5.1.2. Comparison protocol

Every schedule is executed by the same conflict-free router and checked against the composite lower bound by a validator independent of the decoder; failing runs are reported separately. We compare realized makespans, because the open loop's self-reported makespan is often infeasible; the gap (hereafter, the optimism margin) is especially large on the funnel layout. One evaluation costs up to about two orders of magnitude more in some configurations than in others, so configurations share an equal wall-clock budget (Table 5) rather than a generation count, with early stopping after 400 generations without improvement. Every open-loop run stops early, using about one fifth of the budget, while both closed-loop configurations use all of it; this favours the baseline, which completes as many evaluations as closed-loop rule-based dispatching, or more (Section 5.6).

Table 5: Experiment batches; only the main batch supports Tables 7 and 8. Speed-ups: pruning and winner-path reuse (Section 4.5).

Batch	Purpose	Instances	Seeds	Wall-clock budget	Speed-ups
Main	Ladder (Tables 7, 8)	Ten controlled	10	90 s	yes
Cost diagnosis	Figure 4	Three controlled	3	90 s	yes
Convergence diagnosis	Optimism margin (Section 5.3)	`funnel`	3	46.3 s	yes
Speed-up switch	Table 9	Six controlled	10	90 s	off/on
Budget convention	Section 5.6	Four	10	Equal wall-clock; equal generations	no
Surcharge scan	Section 5.5	Two $8 \times 4 \times 4$	10/5	48.9/60.7 s	no
Independent instances	Cross-check, H, ablations (Sections 5.4, 5.5)	16	10	30.2–72.9 s	no
Public benchmarks	Table 10	20 public	10	Early stopping	—

Three seeds on one instance spanned 11 makespan units, more than any mechanism effect, so each instance runs 10 seeds (100 pairs), compared by a two-sided Wilcoxon signed-rank test paired by seed; this resolves a few-percentage-point effect when pooled, not within one instance. Since ten uncorrected tests give at least one false positive with probability about 40%, each family is Holm-corrected: the five pooled contrasts of Table 7, and the 10 instances of each effect in Table 8 (Benjamini–Hochberg disagreements are marked). Two marked exceptions stay uncorrected: the vehicle-to-arm-ratio p-values, non-significant already, and the significant-instance counts of the independent 16-instance batch, used for direction and magnitude only. Pooled contrasts also give a Hodges–Lehmann pseudomedian with a 95% distribution-free interval from the test's null distribution, whereas the tabu-

lated mean pairwise relative gain carries no precision; for dispatching in the closed loop the mean is 2.45% and the pseudomedian 3.03%, because pairs on two vehicle-to-arm-ratio instances worsen. Figure error bars are standard errors of the mean.

5.1.3. Implementation and environment

Experiments run single-threaded on an Intel Xeon W-1270 @ 3.40 GHz in pure Python ($\geq$ 3.9.6) without a solver, with untuned default hyperparameters shared by all configurations: population 60, crossover 0.8, mutation 0.2, elite 5, binary tournament; local search on the top 10%, at most $L = 5$ critical operations per round, three rounds. Compiled code would raise every configuration's evaluation count, not the cost ratios between them.

5.2. Four-level baseline ladder

Ablations alone cannot be mapped onto the pipeline structures of the literature, while a literature comparison alone would credit the gain of conflict-free routing itself to our mechanisms; the ladder serves both purposes as a 2×2 design (Table 6). **B0** searches on unimpeded shortest-path times and executes the plan with the same router and idle-first dispatching (Eq. (12)); constant-matrix FJSP_PT methods (Meng et al., 2025; Qin et al., 2026; Jafarian-Moghaddam et al., 2026) executed on a real road network, and the framework of Li et al. (2026), fall into this class. **B0+** keeps that schedule and changes only vehicle selection to a trial query on the reservation table, a greedy approximation of the second stage of two-stage structures such as Sun et al. (2023). **B1** takes fitness from conflict-free routing but dispatches by Eq. (12); **B2**, our method, differs from B1 only in vehicle se-

lection. B0+ → B1 changes both factors and cannot measure the closed loop.

Table 6: The 2×2 structure of the four-level ladder. Row or column contrasts change one factor; diagonal contrasts cannot be decomposed.

	Rule-based dispatching	Reservation-table-aware dispatching
Open loop (schedule first, then route; search ignores conflicts)	B0	B0+
Closed-loop planning (fitness from conflict-free routing)	B1	**B2** (ours)

B0 either *replays the plan* with its specified vehicles or reassigns them on site. Replay is stronger (63.00 versus 78.00 on the funnel instance) and matches Li et al. (2026), whose execution only shifts the Gantt chart in time; it is the main report, with on-site reassignment listed alongside.

5.3. Aggregate comparison on the ladder

Table 7 reports the ladder on 100 pairs (10 instances × 10 seeds); every run passed the independent check, and Holm correction over the five contrasts changes no decision. The text states Hodges–Lehmann intervals as reductions; the table shows them as negative values.

With the vehicle-selection rule fixed, the closed loop is significantly better under both rules: B0 → B1 is -18.86% ($p < 10^{-4}$) and B0+ → B2 is -17.75% ($p < 10^{-4}$), so the conclusion does not depend on our dispatching rule. For the former the pseudomedian is 18.28% with interval [14.68%, 21.95%]; even its lower end is a two-digit reduction. The number of evaluations runs the other way: looking up a constant matrix, the open loop completes several times as many evaluations within the same budget

Table 7: Four-level ladder under an equal wall-clock budget, and the four one-factor contrasts. Negative is better. $n = 100$ pairs on (instance, seed); two-sided Wilcoxon signed-rank, Holm-corrected over the five contrasts ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$). HL: Hodges–Lehmann pseudomedian with 95% interval. Bold marks our configuration, not a per-instance winner.

		$C_{\max}$ (time units)			
Instance	Contention	B0	B0^{+}	B1	**B2**
A funnel	31.9%	58.10	53.50	44.70	42.40
A high	30.9%	56.00	51.40	39.90	37.70
A low	11.8%	64.50	64.60	57.80	57.10
A mid	10.8%	46.40	42.70	39.40	37.40
A scatter	6.6%	62.30	62.30	57.20	57.10
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20
C F=0.3	27.7%	74.30	69.50	51.60	47.30
C F=1.0	33.5%	52.10	47.50	39.30	36.30
Pooled mean	—	61.20	58.27	48.86	47.44

Single-factor paired contrast	$\Delta C_{\max}$	HL pseudomedian and 95% CI (%)	W/L/T	p_{Holm}
Closed-loop value (rule-based)	$-18.86\%^{***}$	-18.28 $[-21.95, -14.68]$	96/2/2	$< 10^{-4}$
Closed-loop value (reservation-table-aware)	$-17.75\%^{***}$	-18.00 $[-20.55, -15.22]$	93/5/2	$< 10^{-4}$
Dispatching value (open-loop)	$-4.59\%^{***}$	-4.45 $[-6.13, -2.88]$	68/19/13	$< 10^{-4}$
Dispatching value (closed-loop)	$-2.45\%^{**}$	-3.03 $[-5.19, -0.94]$	60/34/6	0.0054
End-to-end: literature → this paper	$-21.35\%^{***}$	-21.20 $[-24.46, -17.99]$	94/5/1	$< 10^{-4}$

(Figure 4) and is still worse, because it optimizes a surrogate that excludes yielding. On a `funnel` diagnostic run (Table 5) it converges, but to a surrogate value its plan cannot realize after execution: the optimism margin.

With the loop fixed, reservation-table-aware dispatching inside the closed loop gives B1 $\rightarrow$ B2 of -2.45% ($p = 0.0054$; pseudomedian 3.03%, interval [0.94%, 5.19%]), with search and budget shared. This is a pooled result: the interval excludes zero by less than one percentage point, no instance is significant after correction (2 before), and at vehicle-to-arm ratios 1.0 and 2.0 B2 is slower than B1 (Table 8). -2.45% is therefore an average over this instance family, not an instance-level estimate. Under the open loop the same rule gives -4.59% ($p < 10^{-4}$), likewise significant; it measures the step from the open-loop to the two-stage structure and is far smaller than the closed loop's 18.86%. Dispatching is thus significant under both loops but much smaller, and cannot by itself explain the end-to-end reduction.

End to end, and how the two effects combine.. Pooled, B0 $\rightarrow$ B2 is -21.35% ($p < 10^{-4}$; 94 wins, 5 losses, and 1 tie among 100 pairs), about one fifth of the makespan. The effects have different reference points and cannot be added directly: were the mechanisms independent, the compound of 18.86% (closed loop, rule-based dispatching) and 4.59% (dispatching, open loop) would be 22.6%, and the measured 21.35% is slightly below it. Each mechanism contributes on its own and *the two do not amplify each other*; B2 is best end to end because the gains add up. This also locates the shortfall of "schedule first, then resolve conflicts": second-stage dispatching gains only 4.59%, far less than 18.86%, so the shortfall lies in the first stage ignoring yielding, and the closed-loop reduction requires yielding wait in the fitness during the

search. These gaps were measured on our instance families.

5.4. Three-factor scans and per-instance resolution

The three families test whether the gains grow with contention intensity, whether trial-query overhead, growing with the fleet while the pruning of Section 4.5 weakens, caps the dispatching gain, and whether the gains need room for reassignment. With Holm correction over the 10 instances of one effect (all marks in Table 8 and Figure 3), the closed loop remains significant on 9 instances (9 uncorrected; largest corrected value $p = 0.0401$), all but the instance with the fewest vehicles (-2.0%, not significant), so it can be read by family. Dispatching is significant on 2 instances before correction, the two ends of the flexibility family ($p = 0.0105$, $p = 0.0321$), and on 0 after it (Holm $p = 0.1052$, $p = 0.2885$; Benjamini–Hochberg $p = 0.1052$, $p = 0.1603$): its few-percentage-point effect is near the smallest that 10 seeds resolve on one instance, and only the 100 pooled pairs separate it from noise, and only narrowly. Its per-instance results are therefore directions only.

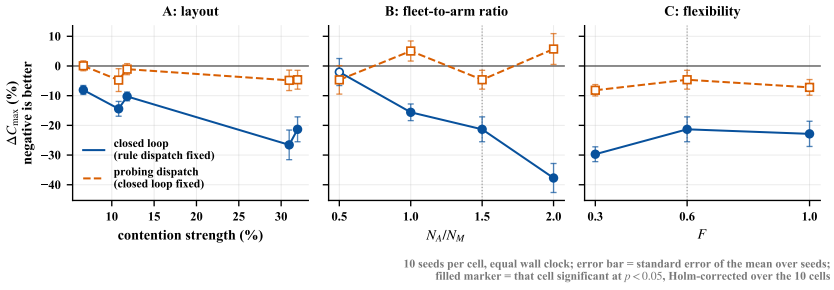


Figure 3: Closed-loop and dispatching gains by family: layout against contention intensity (left), N_A/N_M (middle), flexibility (right); vertical lines mark the base point `A funnel`. Filled markers: significant after Holm correction over the 10 instances of one effect. Error bars: standard error over 10 seeds.

In the layout family the closed loop is significant on all five instances

Table 8: Per-instance ladder results and the two effects, by family ($n = 10$ seed pairs per instance). Marks: Holm-corrected two-sided Wilcoxon over the 10 instances of one effect, $^{*}p < 0.05$, $^{**}p < 0.01$; † significant only before correction. Negative is better; gains are means of pairwise relative gains, as in Table 7. `A funnel` also serves as $N_A/N_M = 1.5$ and $F = 0.6$.

		$C_{\max}$ (time units)				Pairwise relative gain	
Instance	Contention	B0	B0^{+}	B1	**B2**	B0→B1	B1→B2
A funnel	31.9%	58.10	53.50	44.70	42.40	−21.3%*	−4.6%
A high	30.9%	56.00	51.40	39.90	37.70	−26.6%*	−4.8%
A low	11.8%	64.50	64.60	57.80	57.10	−10.3%*	−1.1%
A mid	10.8%	46.40	42.70	39.40	37.40	−14.4%*	−4.8%
A scatter	6.6%	62.30	62.30	57.20	57.10	−8.1%*	+0.1%
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10	−2.0%	−4.7%
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80	−15.6%*	+5.0%
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20	−37.7%*	+5.7%
C F=0.3	27.7%	74.30	69.50	51.60	47.30	−29.7%*	−8.2%†
C F=1.0	33.5%	52.10	47.50	39.30	36.30	−22.9%*	−7.2%†

and rises roughly with contention, from -8.1% on `scatter` (6.6%) through -14.4%/-10.3% on `mid`/`low` to -21.3%/-26.6% on `funnel`/`high`. Two inversions qualify this: `low` versus `mid` straddles the topological boundary, but `funnel` versus `high` is the planned single-factor contrast, so *this inversion is a counter-example*. What is established is the magnitude, about two and a half times larger on the most congested instance than on the least, not a monotone relation between contention and gain; contention intensity does not separate the congestion that reassignment can bypass (Limitation (1), Section 6.2). Dispatching is significant on no instance here; four of five signs match the closed loop.

The vehicle-to-arm-ratio family does not establish an upper bound for dispatching: from $N_A/N_M = 0.5$ to 2.0 it is −4.7%, +5.0%, -4.6%, and

+5.7%, alternating in sign and non-significant even before correction ($p = 0.4121$, $p = 0.2011$, $p = 0.1386$, $p = 0.3313$). Without the base point the curve would show the rise predicted by the cost law, so the upper bound of Section 5.6 remains untested; testing it needs more seeds than our budget allows. The closed loop stays significant at 1.0 and 2.0 (-15.6%, -37.7%): fleet size mainly affects the closed-loop gain.

In the flexibility family dispatching is -8.2%, -4.6%, and -7.2%, all with the pooled sign; the two ends are the largest magnitudes but lose significance after correction. Since $F = 0.3$, with the least room for reassignment, has the largest magnitude, dispatching appears to act through vehicle selection and timing rather than reassignment. The closed loop is -29.7% and -22.9%, significant after correction.

Independent batch and heterogeneity.. A separate batch of contended self-generated instances (1280 runs; Table 5) crosses four contention levels with four H levels against a two-stage surrogate-transport baseline. Its *integration gain* (two-stage $\rightarrow$ closed loop), 5.52%, significant on 11 instances (uncorrected), matches the closed loop here in direction and magnitude. By H level it is 6.12% at $H = 0$, 5.62% at 0.15 (measured 0.139, near the public median 0.135), 5.95% at 0.3, and 4.38% at 0.5, with a weakly negative rank correlation -0.21. Setting H above the literature range thus did not raise the gain; with four instances per level and a within-level spread (0.28% to 9.28%) far exceeding the gaps between levels, the data support only that the gain does not rise with H, not that $H = 0.5$ is lower. As Section 3.7 predicts, a larger H lets processing speed rather than traffic decide assignment, while $H = 0$ makes reassignment a pure congestion decision and has the highest gain; H

creates the trade-off behind the reversal of Section 1 but does not amplify the closed-loop gain. For dispatching this batch gives -0.74% (0 instances significant), opposite to the main batch; it ran before the two speed-ups (Section 5.6).

5.5. Ablations and controlled negative controls

The surcharge is off by default ($\theta = 0$), and yield-record-guided local search runs with the same setting in all four configurations, so neither affects the comparisons between them.

On the 16-instance batch of Section 5.4, valid here because the cost of these operators, unlike that of dispatching, does not depend on pruning, turning off yield-record feedback gives a relative gain of -1.72% (significant on 1 instance) and turning off staggering gives -0.16% (significant on 2). Neither is positive: under an equal wall-clock budget, these decodes are better spent on population evolution. The local search consumes a large share of all decodes, yet fewer than one in fifty of its neighbours improve makespan although the neighbourhood is small and well targeted (Section 4.6), and three refinements of its operators were each not significant under an equal number of evaluations, so the signal itself fails.

The surcharge counts again a delay that fitness already includes (Section 4.6), so it adds no information, but its cost must be measured. On two instances outside Families A/B/C (Table 5), with the path-planning search width fixed, every surcharge coefficient θ tried worsens makespan significantly at equal wall-clock time, flat over a ten-fold range, whereas at an equal number of evaluations the difference cannot be distinguished from zero. Routing with a surcharge completes only about one fifth as many evaluations, because

a second criterion removes total-order dominance (Lemma 1) and requires a multi-label search whose overhead is the same at every θ. Nor is θ a useful tuning parameter: the cumulative surcharge is too small to reorder arrivals, so makespan is unchanged over two orders of magnitude until the surcharge outweighs the arrival time and the double counting worsens it sharply. An interface that passes a quantity already contained in the objective of the receiving layer is therefore harmful under our protocol.

Dilution and overhead.. A yield record names what is needed to remove one corridor delay, but makespan is set by a chain: once that delay is removed, another constraint usually becomes tight, so makespan improves by far less than the yielding time removed. The same dilution leaves reservation-table-aware dispatching only a few percentage points, lost in noise at a cost multiple of 12.2 and significant in the pooled data once Section 4.5 lowered it to 5.23. Local search thus fails because too little reduction survives dilution, whereas dispatching initially failed because each evaluation cost too much; dispatching is therefore a mechanism contribution, and yield-record-guided local search only a controlled negative control.

5.6. The cost law and the effect of the two speed-ups

Under a fixed wall-clock budget, the cost per evaluation and the number of evaluations trade off against each other (Figure 4): the cost spans about two orders of magnitude, and the open loop completes the most evaluations in about one fifth of the wall-clock time. An equal number of generations lets a costlier configuration use more compute and asks whether extra routing improves the ranking within a generation; equal wall-clock time asks whether

makespan falls within the same time. The two conventions disagree: before the speed-ups, reservation-table-aware dispatching is 2.90% ($p = 0.0269$) at equal generations but only 0.03% ($p = 0.8799$) at equal wall-clock time, so we report both.

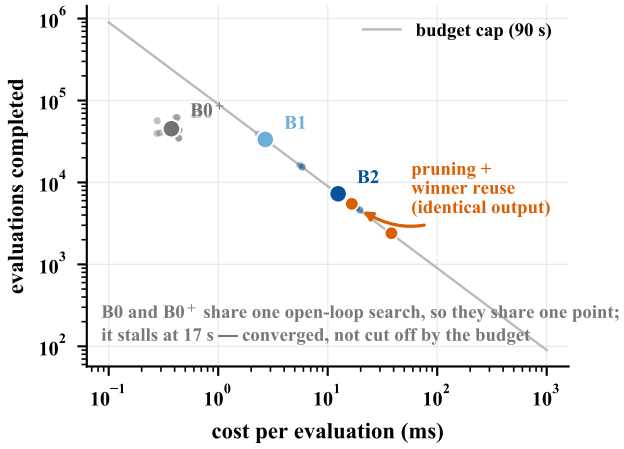


Figure 4: Cost per evaluation against evaluations completed under the shared 90-second cap (grey hyperbola). The closed-loop configurations use the full budget; the open loop (B0 and B0$^+$ share one point) is ended by early stopping after about 17 s. Pale points: three instances; filled points: their geometric mean. Orange: pruning and winner-path reuse off and on.

The cost law follows from the trial query: it raises the routing calls per transport task from 2 to at most $2N_A$, so cost grows linearly with fleet size, while pruning cuts fewer calls as fleet and contention grow (61.2% on the least congested self-check instance, 42.2% on the largest fleet). The law predicts a fleet size beyond which the trial-query cost offsets the gain; across the four vehicle-to-arm levels of Section 5.4 the sign alternates and no instance is significant, so the law is measured but this bound is not confirmed.

Because the dispatching rule and the speed-ups were introduced one after the other, a separate control changes only pruning and winner-path reuse,

with the same instances, seeds and budget. They change neither the selected vehicle nor the timetable (Proposition 3): for six random chromosomes on each of six instances, makespans and dispatch sequences are identical bit for bit in all 36 cases while routing calls fall by 47.4%, so any difference comes only from the compute saved. Switching them on lowers makespan by a further 5.60% ($p < 10^{-4}$; Table 9). The same rule thus moves from -0.74% before the speed-ups (0 of the 16 instances significant) to significantly positive after them, without any change of selection.

Table 9: Pruning and winner-path reuse off versus on (bold: adopted setting). Same instances, seeds and wall-clock time; $n = 60$ pairs, two-sided Wilcoxon signed-rank test ($^{***}p < 0.001$). Ratios take the off value as denominator.

		$C_{\max}$ (time units)			Eval. count ratio	Speedup per eval.
Instance	Contention	Off	**On**	$\Delta C_{\max}$	(on/off)	(off/on)
A funnel	31.9%	44.80	42.10	−6.03%	2.21×	2.23×
A high	30.9%	41.60	37.70	−9.37%	2.40×	2.41×
A low	11.8%	60.50	57.00	−5.79%	2.79×	2.81×
B NA/NM 0.5	15.8%	66.70	65.10	−2.40%	2.11×	2.11×
B NA/NM 1.0	30.2%	51.90	48.90	−5.78%	2.31×	2.31×
B NA/NM 2.0	32.4%	44.50	42.20	−5.17%	1.98×	2.01×
Pooled (60 pairs)	—	—	—	−5.60%***		50/0/10, $< 10^{-4}$
HL pseudomedian and 95% CI: -4.72 $[-6.34, -3.55]$ %						

5.7. Case study: Gantt chart and critical chain on the funnel instance

Aggregate numbers do not show what happens inside one solve, so Figure 5 places B0 and B2 side by side on one funnel instance. In the open-loop solution, yielding waits cluster on the corridors near the load/unload point v_0 and on a few vehicles. The closed-loop solution greatly reduces them and assigns several operations to an arm that is not the fastest (Eq. (14)), trading a longer processing time for a less congested path; its more even vehicle

load follows from these two effects rather than from a separate mechanism. Labelled as in Table 3, the B0 critical chain contains a substantial share of `corridor` links; on B2 they largely give way to `machine` and `upstream` links, the dilution of Section 5.5.

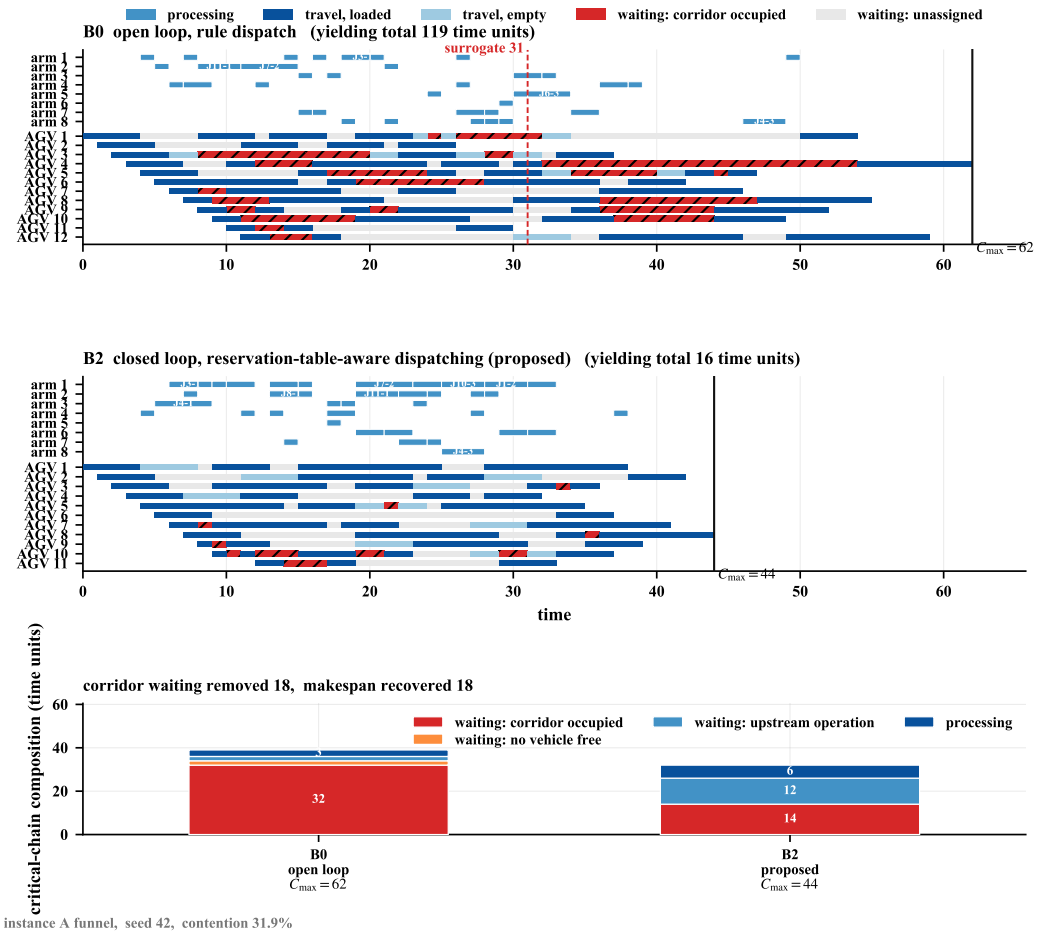


Figure 5: B0 and B2 on one funnel instance. Top: arms and vehicles on a shared time axis, with empty and loaded legs; a gap between the two legs of one transport is a yielding wait at a berth, a gap between tasks is idle time. Bottom: critical-chain composition by the labels of Table 3, with corridor wait removed and makespan reduced.

5.8. Degenerate comparison on public benchmarks

Comparisons between configurations of one implementation cannot calibrate its absolute level. Without the exclusivity constraint, Eq. (8), and with $\delta_{\text{ret}} = 0$ (Table 2), our model is exactly FJSP_PT (Section 3), the *same mathematical problem* as the public benchmarks; neither mechanism is then active, so this anchors the upper-level search, not the framework. We use the 20 instances of Homayouni and Fontes (2021) (from Fattahi et al. (2007)) with two vehicles as in Lim and Moon (2023), and their published travel matrices as t^*, since 2 are not a shortest-path closure (Section 5.1); lacking a road network, they admit only this degenerate setting. References are MILP optima for 16 instances (Homayouni and Fontes, 2021) and best published values for 4 (Lim and Moon, 2023).

All 200 runs pass validation and never fall below the composite lower bound or a proven optimum. On 12/20 instances, including all 10 small ones, the best solution equals the reference digit for digit (Table 10); the mean gap is 2.68%, rising to 9.96% on the four largest. This reflects search capability, not budget: 2.5 times the wall-clock time lowers the mean gap of the 10 MFJST instances only from 5.37% to 4.53%, and all runs end by early stopping. On the proven-optimum instances the bound lies a median 10.3% below the optimum while the search is at most 4.1 percentage points above it, so without contention the bound gap is almost entirely bound slack. Since contention can only raise the optimum, the median bound gap of 42% on the self-generated instances *cannot* be read as a search gap.

Table 10: Degenerate comparison on public benchmarks (exclusivity removed). OPT: MILP-proven optimum; BK: best published value; gap from the best of 10 seeds; asterisk: identical to the reference.

			$C_{\max}$ (time units)			
Instance	Ops	Ref.	Value	Best	Mean	Gap
SFJST01	4	OPT	70	70*	70.0	0.00%
SFJST02	4	OPT	111	111*	111.0	0.00%
SFJST03	6	OPT	223	223*	223.0	0.00%
SFJST04	6	OPT	359	359*	359.0	0.00%
SFJST05	6	OPT	123	123*	123.0	0.00%
SFJST06	9	OPT	324	324*	324.0	0.00%
SFJST07	9	OPT	409	409*	409.0	0.00%
SFJST08	9	OPT	269	269*	269.5	0.00%
SFJST09	9	OPT	220	220*	222.0	0.00%
SFJST10	12	OPT	531	531*	531.0	0.00%
MFJST01	15	OPT	485	485*	485.0	0.00%
MFJST02	15	OPT	463	476	483.4	2.81%
MFJST03	18	OPT	482	482*	504.7	0.00%
MFJST04	21	OPT	576	586	623.0	1.74%
MFJST05	21	OPT	532	561	592.8	5.45%
MFJST06	24	OPT	652	677	728.3	3.83%
MFJST07	32	BK	898	965	1023.3	7.46%
MFJST08	36	BK	900	970	1027.0	7.78%
MFJST09	44	BK	1098	1226	1279.2	11.66%
MFJST10	48	BK	1230	1389	1473.0	12.93%

6. Conclusion

When transport takes longer than processing and congestion concentrates on a few corridors, assignment and conflict-free AGV routing must be decided together. Our closed-loop bilevel framework feeds routing information into decisions through an evaluation path and a decision path, and measures each under the same wall-clock budget.

6.1. Findings and contributions

1. **Instance families with controllable contention intensity.** Varying layout, vehicle-to-arm ratio and flexibility, they show a closed-loop gain that broadly rises with contention, to about two and a half times on the most congested instance, but not monotonically.

2. **Routing-in-the-loop evaluation.** Every chromosome is decodable and evaluated consistently with the conflict model, without repair (Proposition 2, Lemma 1); the closed loop reduces makespan by 18.86% ($p < 10^{-4}$) under rule-based and by 17.75% ($p < 10^{-4}$) under reservation-table-aware dispatching.

3. **Reservation-table-aware dispatching with output-preserving speed-ups.** A trial query of both legs reduces makespan by a further 2.45% ($p = 0.0054$); pruning and winner-path reuse change neither vehicle nor timetable (Proposition 3) and lower the cost multiple from 12.2, at which the gain is lost, to 5.23.

4. **The two paths differ in magnitude and are not substitutes.** The closed loop is significant after Holm correction on 9 of the 10 instances,

dispatching only when pooled (4.59%, $p < 10^{-4}$, open loop; interval [0.94%, 5.19%], closed loop). The end-to-end gain 21.35% is slightly below the independent-composition prediction 22.6%, and scheduling before resolving conflicts forgoes the 18.86% of closed-loop evaluation.

6.2. Limitations and future work

(1) Instance scope. Size and $H = 0.3$ are fixed (Section 3.7), so we claim nothing about size, and "the gain does not rise with H" holds only for the smaller batch of Section 5.4; only the three spine instances vary capacity alone, since the grids also change topology and distance (farthest leg 10.0 versus 18.0), and contention intensity does not separate the yielding that reassignment can bypass. The instances are self-generated because no public benchmark has an explicit road network. Future work should remeasure on larger corridor networks, treating bypassability by reassignment and a single dominant bottleneck as separate factors under a new, validated measure.

(2) Stability of the dispatching gain. Its effect of a few percentage points is of the order of the dispersion across seeds: 2 of 10 instances are significant before correction and 0 after, so it is confirmed positive only over the 100 pooled pairs (interval [0.94%, 5.19%]). About ten times more seeds and denser vehicle-to-arm levels would locate where it is stable and test the fleet-size bound of the cost law.

(3) Distance from the optimum under contention. Search strength is common to all configurations, so their differences are unaffected; without contention, the four largest instances average 9.96%, gaining only 0.84 percentage points from 2.5 times the computation, with a seed-to-seed standard deviation of up to 2.3%–4.5% of the mean, so the encoding or neighbour-

hood needs improvement. With contention no optimum is known, and an effect such as 2.45% may be of the order of the distance to it; exact solutions by logic-based Benders decomposition (van Os and Basten, 2026) on small contended instances would separate bound slack from insufficient search and retest the two negative controls under a stronger search.

(4) Relative cost of the negative controls. The surcharge and yield-record-guided local search were measured as a trade-off between cost per evaluation and number of evaluations: a uniform speed-up keeps the sign, but cutting the cost of one interface alone could change it, as for dispatching. Since the decoder state advances in one direction, re-solving only the part after a perturbation would lower the cost of local search, which should then be remeasured.

(5) Static setting. Assumption A7 excludes robotic-arm and vehicle failures and dynamic job arrivals, so the results hold for a known job set and failure-free equipment. Under such disruptions, studied for the dynamic flexible job shop (Gui et al., 2023; Zhao and Fu, 2026; Dauzère-Pérès et al., 2024), a mechanism is also valued by how fast it repairs a schedule, which we do not measure, and whether re-solving or local repair is preferable cannot be inferred from the static setting.

References

Dauzère-Pérès, S., Ding, J., Shen, L., Tamssaouet, K., 2024. The flexible job shop scheduling problem: A review. European Journal of Operational Research 314, 409–432. doi:10.1016/j.ejor.2023.05.017.

Fattahi, P., Saidi Mehrabad, M., Jolai, F., 2007. Mathematical modeling and

heuristic approaches to flexible job shop scheduling problems. Journal of Intelligent Manufacturing 18, 331–342. doi:10.1007/s10845-007-0026-8.

Fragapane, G., de Koster, R., Sgarbossa, F., Strandhagen, J.O., 2021. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. European Journal of Operational Research 294, 405–426. doi:10.1016/j.ejor.2021.01.019.

Gao, J., Li, Y., Li, X., Yan, K., Lin, K., Wu, X., 2023. A review of graph-based multi-agent pathfinding solvers: From classical to beyond classical. Knowledge-Based Systems 283, 111121. doi:10.1016/j.knosys.2023.111121.

Gui, Y., Tang, D., Zhu, H., Zhang, Y., Zhang, Z., 2023. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. Computers & Industrial Engineering 180, 109255. doi:10.1016/j.cie.2023.109255.

Homayouni, S.M., Fontes, D.B.M.M., 2021. Production and transport scheduling in flexible job shop manufacturing systems. Journal of Global Optimization 79, 463–502. doi:10.1007/s10898-021-00992-6.

Jafarian-Moghaddam, A.R., Esmailpoor Sanjabi, A., Mortazavi Moghaddam, E., 2026. Integrated tri-objective scheduling and internal logistics optimization in U-shaped flexible flow shops: a mathematical model and metaheuristic framework. International Journal of Systems Science: Operations & Logistics 13, 2714000. doi:10.1080/23302674.2026.2714000.

Kasaura, K., Nishimura, M., Yonetani, R., 2022. Prioritized safe interval path planning for multi-agent pathfinding with continuous time on 2D roadmaps. IEEE Robotics and Automation Letters 7, 10494–10501. doi:10.1109/LRA.2022.3187265.

Kim, J., Kim, J., 2026. Congestion-aware scheduling for large fleets of AGVs using discrete event simulation. Electronics 15, 139. doi:10.3390/electronics15010139.

Li, R., Mao, J., Wu, X., Zhou, W., Qian, C., Du, H., 2026. A new framework for job shop integrated scheduling and vehicle path planning problem. Sensors 26, 543. doi:10.3390/s26020543.

Lim, C.H., Moon, S.K., 2023. A two-phase iterative mathematical programming-based heuristic for a flexible job shop scheduling problem with transportation. Applied Sciences 13, 5215. doi:10.3390/app13085215.

Lu, W., Guo, S., Song, T., Li, Y., 2022. Analysis of multi-AGVs management system and key issues: A review. Computer Modeling in Engineering & Sciences 131, 1197–1227. doi:10.32604/cmes.2022.019770.

Lyu, X., Song, Y., He, C., Lei, Q., Guo, W., 2019. Approach to integrated scheduling problems considering optimal number of automated guided vehicles and conflict-free routing in flexible manufacturing systems. IEEE Access 7, 74909–74924. doi:10.1109/ACCESS.2019.2919109.

Meng, L., Cheng, W., Zhang, C., Gao, K., Zhang, B., Ren, Y., 2025. Novel CP models and CP-assisted meta-heuristic algorithm for flexible job shop

scheduling benchmark problem with multi-AGV. IEEE Transactions on Systems, Man, and Cybernetics: Systems 55, 8455–8468. doi:10.1109/TSMC.2025.3604355.

van Os, R.W.M., Basten, T., 2026. An exact decomposition-based approach to the conflict-free-transportation-constrained flexible job-shop scheduling problem. Computers & Operations Research 187, 107342. doi:10.1016/j.cor.2025.107342.

Otala, J., Minard, A., Madraki, G., Mousavian, S., 2021. Graph-based modeling in shop scheduling problems: Review and extensions. Applied Sciences 11, 4741. doi:10.3390/app11114741.

Qin, H., Xiang, Y., Han, Y., Wang, Y., Li, J., Pan, Q., 2026. A knowledge region selection enhanced quality-diversity algorithm for real-world flexible job shop scheduling with automated guided vehicles transportation. Engineering Applications of Artificial Intelligence 164, 113352. doi:10.1016/j.engappai.2025.113352.

Rahimikelarijani, B., Fazlollahtabar, H., Nayeri, S., 2020. Multi-objective multi-load tandem autonomous guided vehicle for robust workload balance and material handling optimization. SN Applied Sciences 2, 1032. doi:10.1007/s42452-020-3002-y.

Suh, Y.J., Choi, J.Y., 2022. Efficient Fab facility layout with spine structure using genetic algorithm under various material-handling considerations. International Journal of Production Research 60, 2816–2829. doi:10.1080/00207543.2021.1904159.

Sun, J., Xu, Z., Yan, Z., Liu, L., Zhang, Y., 2023. An approach to integrated scheduling of flexible job-shop considering conflict-free routing problems. Sensors 23, 4526. doi:10.3390/s23094526.

Tamssaouet, K., Dauzère-Pérès, S., 2023. A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems. European Journal of Operational Research 311, 455–471. doi:10.1016/j.ejor.2023.05.018.

Xin, B., Lu, S., Wang, Q., Deng, F., 2025. A review of flexible job shop scheduling problems considering transportation vehicles. Frontiers of Information Technology & Electronic Engineering 26, 332–353. doi:10.1631/FITEE.2300795.

Zhang, F., Li, R., Gong, W., 2024a. Deep reinforcement learning-based memetic algorithm for energy-aware flexible job shop scheduling with multi-AGV. Computers & Industrial Engineering 189, 109917. doi:10.1016/j.cie.2024.109917.

Zhang, Y., Jiang, H., Bhatt, V., Nikolaidis, S., Li, J., 2024b. Guidance graph optimization for lifelong multi-agent path finding, in: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI). doi:10.48550/arXiv.2402.01446.

Zhao, F., Fu, Z., 2026. A deep reinforcement learning algorithm with heterogeneous graph and hierarchical attention mechanism for dynamic flexible job shop scheduling problem. Engineering Applications of Artificial Intelligence 183, 116192. doi:10.1016/j.engappai.2026.116192.